Sort Comparison Visualizer

개발자 : 정유운 [20231055]

Sort Comparison Visualizer

< 1. 개요 >

(1) 프로젝트 목적

삽입 정렬, 퀵 정렬, 버블 정렬을 시각적으로 표현하고 연산 횟수를 표기함으로써 자료구조를 수강하고 있는 학생들이 보다 더 해당 알고리즘을 이해할 수 있도록 한다.

Sort Comparison Visualizer

< 1. 개요 >

(2) 어플리케이션 구성

사용 언어 : Python 3.14

주요 라이브러리 : Tkinter, Random

파일 구성 : 1. main.py
2. Services 폴더 (GUI, OP, Con Manager)
3. Templates 폴더 (Sort, GUI Module)

Sort Comparison Visualizer

< 1. 개요 >

(3) 주요 기능

1. 데이터의 값 크기 지정
2. 데이터 집합의 크기 지정
3. 초기 데이터 유형, 정렬 속도 조정
4. 자동 정렬 및 단일 정렬 연산

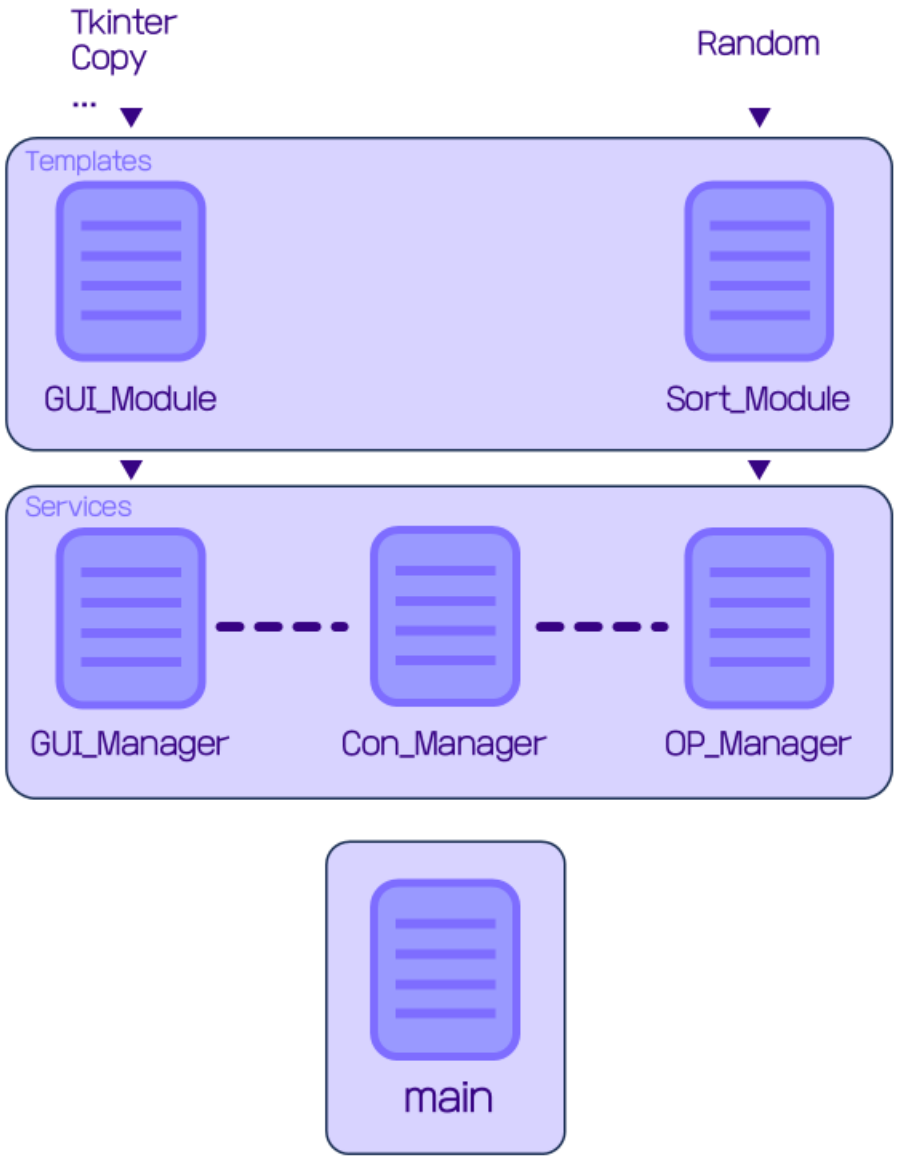
Sort Comparison Visualizer

< 1. 개요 >

(4) 운영 및 유지 보수

해당 어플리케이션은 GitHub를 통하여 베포하고 있으며 해당 프로젝트안에 파일들을 다운로드 하여 어플리케이션을 사용할 것

[GitHub Link] - https://github.com/Ovie-hex/Software-Project-P2-Sort1055



[시스템 구조]

1. Main

(1) main.py
- Services에 있는 모든 Service를 불러오고 GUI를 생성한다

2. Services

(1) GUI_Manager
- GUI 객체 생성, 사용 및 이벤트 객체 사용

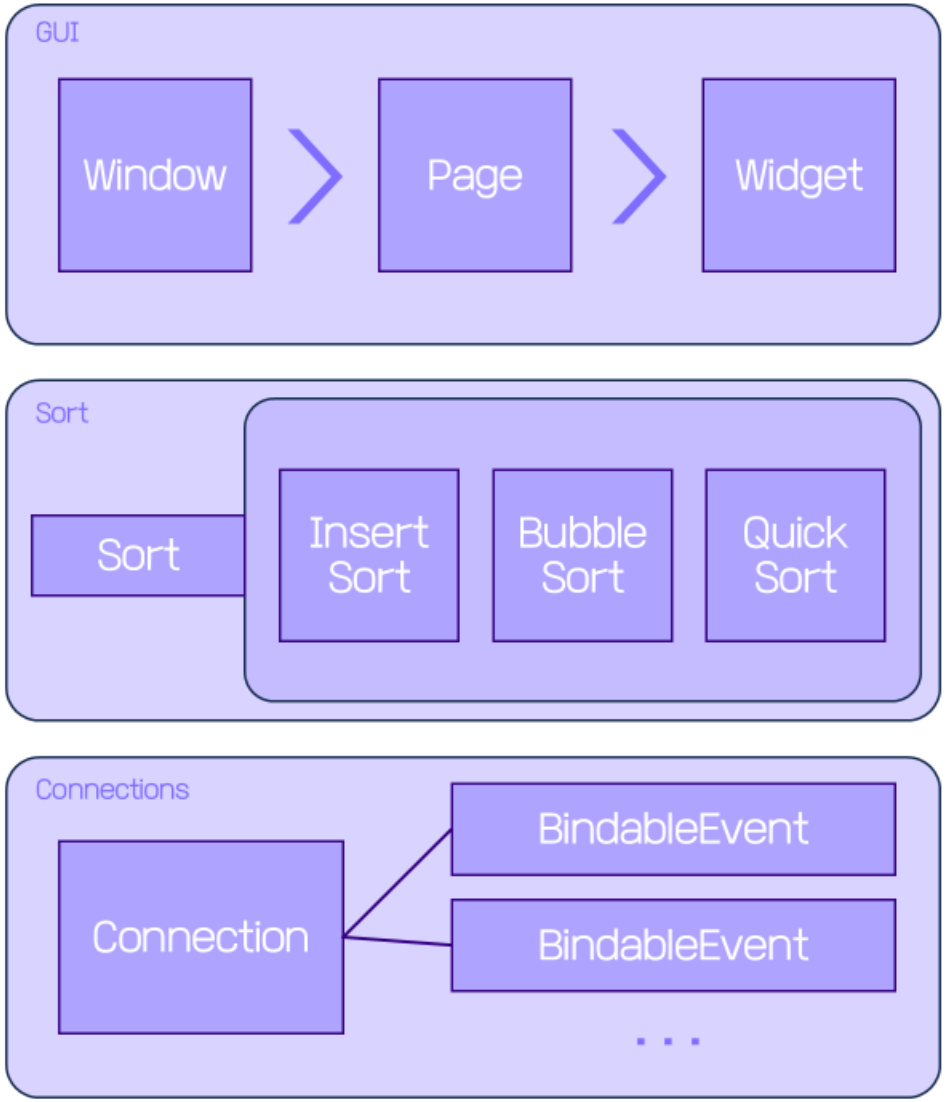
(2) OP_Manager
- 프로그램 주요 기능 처리 및 이벤트 객체 사용

(3) Con_Manager
- 이벤트 객체 생성 및 정의

3. Templates

(1) GUI_Module : GUI 객체 정의

(2) Sort_Module : Sort(정렬) 객체 정의



[자료 구조]

1. GUI (GUI_Module)

(1) Window : Page 클래스 관리 객체
(2) Page : Widget 클래스 관리 객체
(3) Widget : Tkinter의 주요 위젯을 감싼 객체

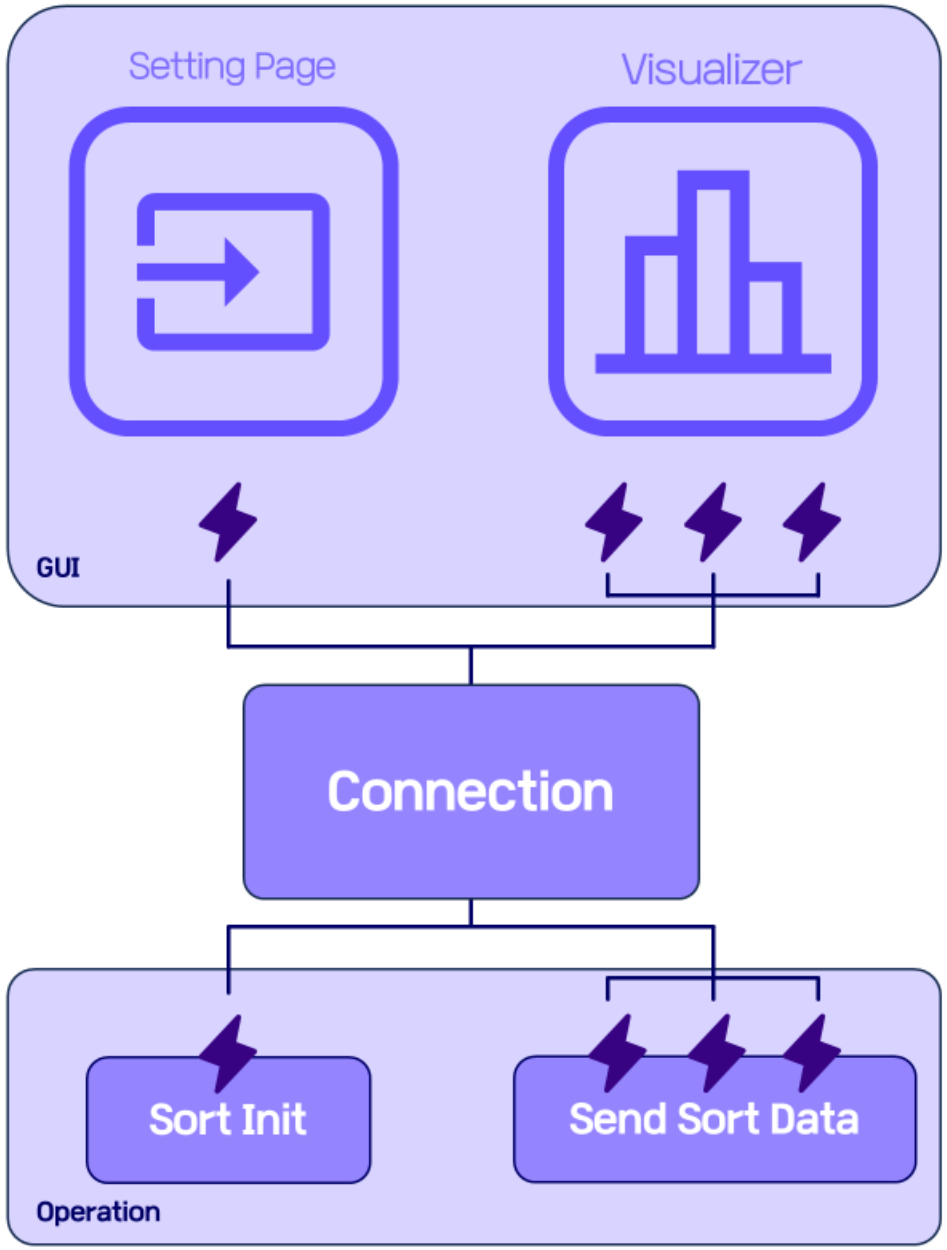
2. Sort (Sort_Module)

(1) Sort : <추상 클래스>
- 리스트 데이터와 정렬하는데 쓰이는 기본 맴버 변수, 멤버 함수(act)를 저장한다.

(2) Insert, Bubble, Quick Sort
- Sort의 인터페이스를 상속한 클래스로 act 함수를 오버라이딩 하여 정렬 기능을 수행함과 동시에 Op_Manager로 GUI의 기능을 수행하는 명령어를 인출한다.

3. Connections (Con_Manager)

- <5. 핵심 기술 >에서 내용 설명



[핵심 알고리즘]

1. 초기 값 전달을 통한 초기 데이터 생성 및 전달
   - 데이터 개수, 무작위 값 크기, 데이터의 상태를 입력하고 그 데이터들을 정리한 리스트를 Operation으로 전달한다
   - Operation 로직에서 유효성을 검증 한 수 초기 데이터를 생성, 해당 데이터를 GUI로 전달한다.

2. 정렬 알고리즘 수행 및 해당 처리 내역 생성
   - GUI 버튼을 통하여 정렬 프로세스를 실행하고 해당 처리 내역을 아래와 같은 데이터 형식으로 기록한다.
   - Ex) [[“comp”, 1, 2], [“swap”, 1, 2]]

3. 알고리즘 처리 내역을 GUI에 전달 후 그래픽 반영
   - 해당 데이터를 GUI로 언패킹하여 프로세스에 따라서 정렬 그래프의 상태 혹은 위치를 반영한다

< gui manager >

getSortLog()
ㄴ 여기에서 처리 모듈을 import 해야하는 상황 발생!
(import op module? manager?)

applyGraph()

< op manager >

initData()
createSortLog() { return log }

op module : 내가 너의 아버지도 아니고...

▼

< con manager >
getBindableEvent(name)

< gui manager >
onlogev = con.getBindableEvent("log")
onlogev.callback = applyGraph()

< op manager >
onlogev = con.getBindableEvent("log")
onlogev.fire(createSortLog())

con manger : 나한테 맡기면 원하는 곳에 전달해줄게!

[BindableEvent]

1. 개요

모듈을 제작할 때 개의치 않게 모듈의 목적과 상관 없는 프로세스를 처리하는 코드를 짜는 경우가 허다하다. 이렇게 되는 경우 특정 모듈의 역할의 중요도가 떨어지고 이른바 스파게티 코드가 될 확률이 높다. 따라서 각 모듈을 완전히 독립시키고자 해당 기술을 사용하였다.

개발자는 이 기술을 만드는데 무려 3일이 걸렸다. (사유 : 하드코딩 + 버그, 정확하게 161줄...)

```
class Connection:
    pass

class BindableEvent:
    pass

class Connection_Manager:
    def getBindableEvent():
        pass
```

[BindableEvent]

2. 설계

해당 기술에서 이벤트를 생성 할때는 고유한 이름을 가진 BindableEvent를 반환하는 함수 getBindableEvent()를 통하여 유일한 이벤트 객체를 모듈에 반환한다.

모듈이 이벤트에 데이터를 전달하게 되면 이벤트들의 집합인 Connection에서 다른 모듈의 이벤트의 필터링 함수, 콜백 함수를 수행한다.

```
class Connection:
    def __init__(self, name, event):
        self._name = name
        self._events = {}
        self._count = 0

        self._events[event] = {...}
        event._connection = self
        self._count += 1

    def addLink():

class BindableEvent:
    def __init__(self, name, ...):
        self.name = hash(name + token)
        self.connection = None
        ...

class Connection_Manager:
    def getBindableEvent(name):
        ev = BindableEvent(name)
        con = next(name에 대한 조건식, None)
        if con:
            con.addlink(ev)
        else:
            con = Connection(name, ev)

        return ev
```

[BindableEvent]

3. 주요 내용 (1) : 이벤트 생성

getBindableEvent()을 통하여 생성하며 매개변수에는 이름이 들어간다.

해당 이름을 가진 Connection이 없는 경우 이벤트 생성 후 새로운 Connection을 생성하여 이벤트를 해당 객체에 할당 시킨다.

만약 Connection이 존재하는 경우 기존 객체에 addLink()를 통하여 이벤트를 할당한다.

```
class Connection:

   def adjustFilter(func):
   def adjustCallback(func):

class BindableEvent:
   @property
   def eventFilter()

   @eventFilter.setter
   def eventFilter(func):
      self._connection.adjustFilter(func)

   @property
   def eventFilter()

   @eventFilter.setter
   def eventCallback(func):
      self._connection. adjustCallback(func)
```

[Module]

```
ev = con.getBindableEvent("Test")
def evFilter(requestorName, data):
def evCallback(requestorName, data):

ev.eventFilter = evFilter
ev.eventCallback = evCallback
```

[BindableEvent]

3. 주요 내용 (2) : Filter, Callback 함수 지정

모듈에서 이벤트 객체를 받고 직접 맴버 변수인 eventFilter, eventCallback을 수정할 수 있으며 Connection 객체의 함수를 통하여 기존 함수에서 새 함수로 덮어씌운다.

이벤트를 생성 할 때 Filter, Callback은 자동으로 클래스 BindableEvent의 정적 메소드인 BindableEvent._nothing을 할당한다. (처음에는 아무것도 없기 때문)

```
class Connection:

    def send(event, data):
        self._event에서 호출자를 제외한 모든 이벤트의
        eventFilter, eventCallback 실행

class BindableEvent:

    def fire():
        self._connection.send(self, data)

[ Module 1 ]

ev = con.getBindableEvent("Test")
ev.fire(data)

[ Module 2 ]

ev = con.getBindableEvent("Test")
def evFilter(requestorName, data):
def evCallback(requestorName, data):

ev.eventFilter = evFilter
ev.eventCallback = evCallback
```

※ 여기서는 예시를 보여줄 뿐입니다. 단일 쓰레드에서 ev생성과 ev.fire()를 동시에 작성 하면 안됨!!!

[BindableEvent]

3. 주요 내용 (3) : Event.fire를 통한 전달

모듈이 특정 모듈로의 데이터 전달을 위하여 이벤트의 fire함수를 사용하는 경우 해당 이벤트 객체는 Connection에 데이터를 맡겨 이벤트의 이름과 일치하지 않는 모든 이벤트의 함수인 eventFilter와 eventCallback을 실행한다.

이때 eventFilter에서 false 값을 반환하는 경우 eventCallback은 실행되지 않는다.

원래 이벤트의 고유 ID를 추가하여 ID 기반으로 비교했어야 했는데 지금 보니 아쉬운 부분이다.

Sort Comparison Visualizer

<6. 결과 데모>

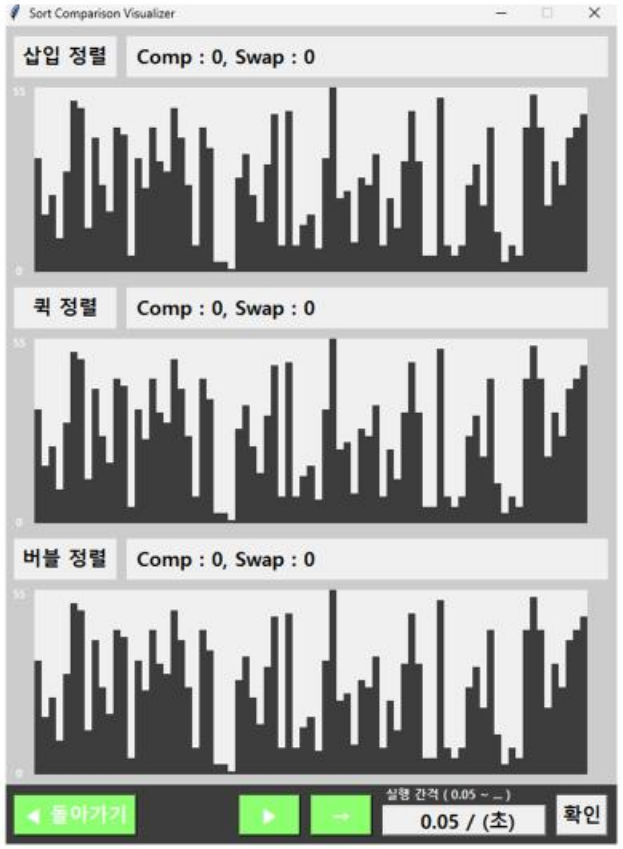
1. 초기 데이터 설정



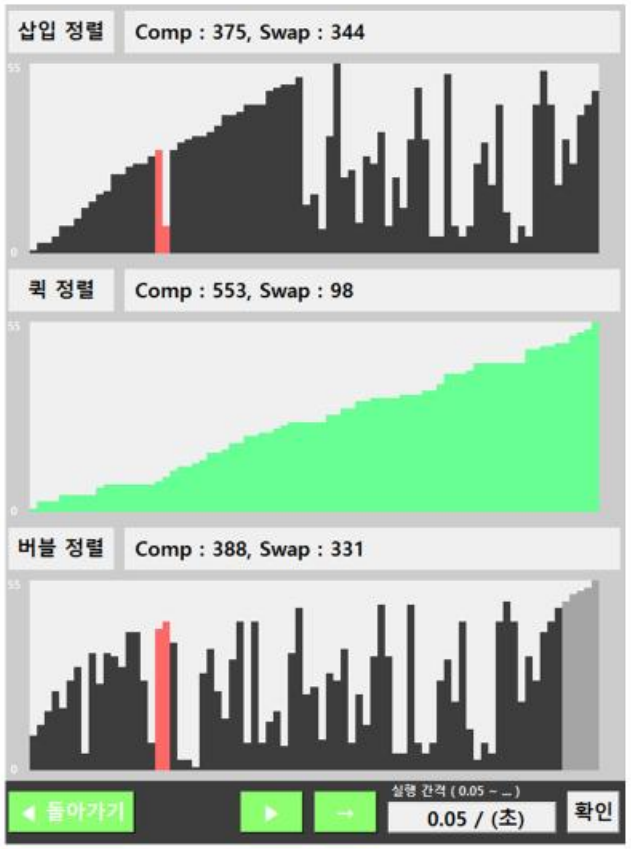
[시연 영상]

Youtube

2. 정렬 초기 화면



3. 실시간 정렬 알고리즘 실행



Sort Comparison Visualizer

< 7. 자체 평가 및 감상 >

(1) 좋은 점 및 아쉬운 점

이 프로젝트를 통하여 프로그램을 개발하고 테스트를 하는 과정에서 각 정렬을 더욱 이해하게 된 것 같아 좋았다. 또 GUI를 구성하는 중 [문제 1] 핵심기술인 (Cascade GUI Wraping)의 설계를 개선하고 경험에 기반한 새로운 기술(BindableEvent)을 만드는 등 지난 결과보다 더욱더 설계 기술을 끌어올렸다고 생각한다.

아쉬운 점은 AI를 생성한 코드를 전혀 사용하지 않고 손으로 전부 작성했지만 결국은 API를 학습하는 것에서 AI를 빠른 완성을 위해서 어쩔 수 없이 사용했다는 점이다. API문서나 인터넷 자료만 보면서 했으면 더욱 좋았을 것 같다.

Sort Comparison Visualizer

< 7. 자체 평가 및 감상 >

(2) 총평

[문제 1] 때와는 다르게 AI를 사용하지 않는다는 원칙이 있었기 때문에 최대한 그것 없이 노력하려고 했으나 결국은 학습을 해서 직접 코드를 짜야 했기에 빠른 학습을 위해 AI를 사용하였다. 다만 학습을 통해 배우고 직접 짜는 코드는 그 만큼 기억에 더욱 생생하게 남기에 결국은 학습을 하더라도 직접 이해하고 개발을 많이 할 수록 그만큼 내가 자신감이 생기고 개발 능력이 향상된다는 것을 깨달았다.

감사합니다!

Youtube link : https://www.youtube.com/shorts/HAdkv73YHak